

Módulo Bilaterales — Parametrización de mapeos

Módulo Bilaterales — Guía de parametrización de mapeos

Propósito: explicar, con base en el código actual, cómo funcionan los mapeos del módulo de bilaterales para poder **parametrizarlo**: SP ↔ proyectos bilaterales de Agresso, indicadores STAR ↔ indicadores ToC, qué endpoints de CLARISA se usan, y qué datos se replican en BD vs. se consultan en caliente.

Fecha: 2026-06-25 · **Rama:** AC-1594-bilateral-module-v2

Todas las referencias `archivo:línea` apuntan a `server/researchindicators/src/`.

0. Mapa rápido del módulo

```
domain/entities/bilateral/                                # núcleo de alineación pool-funding + ToC
├─ bilateral.controller.ts                                # endpoints REST
├─ bilateral.service.ts                                    # lógica (deriveSciencePrograms, validate
├─ entities/
│   ├─ result-pool-funding-alignment.entity.ts            # cabecera de alineación por result
│   ├─ result-pool-funding-alignment-sp.entity.ts         # SP seleccionados
│   ├─ result-pool-funding-indicator-mapping.entity.ts    # contribución por indicador
│   └─ result-pool-funding-toc-alignment.entity.ts        # *** mapeo STAR ↔ ToC ***
└─ utils/toc-level-rules.util.ts                          # niveles ToC permitidos por tipo de ind

domain/entities/bilateral-project-mapping/                # join Agresso ↔ CLARISA project (admin)
└─ entities/bilateral-project-mapping.entity.ts           # *** mapeo proyecto bilateral ↔ Agresso

domain/tools/clarisa/projects/clarisa-projects.service.ts # GET api/projects (en caliente)
domain/tools/toc-integration/toc-integration.service.ts  # GET lambda ToC (en caliente)
domain/tools/prms-toc/prms-toc.service.ts                 # GET PRMS ToC (en caliente)
domain/tools/clarisa/anum/path.enum.ts                    # catálogo de endpoints CLARISA
```

1. Mapeo SP ↔ Proyectos bilaterales de Agresso

1.0 La idea en lenguaje sencillo

Un proyecto de Agresso **no sabe directamente** a qué Science Programs (SP) pertenece. Para averiguarlo, el sistema usa **CLARISA como intermediario**. Por eso hablamos de **dos saltos**: Agresso →

CLARISA → SP .

Analogía. Imagina que quieres saber en qué materias está inscrito un estudiante, pero solo tienes su número de matrícula:

- 1. Primero buscas en una **libreta** la equivalencia "número de matrícula → carné de la universidad". Esa libreta **la llenaste tú a mano** y queda guardada.
- 2. Con ese carné vas al sistema de la universidad **en ese momento** y preguntas "¿en qué materias está?". Eso **no lo tienes guardado**: lo consultas en vivo cada vez.

En nuestro caso:

Analogía	En el módulo bilateral
Estudiante / número de matrícula	Contrato de Agresso
Carné de la universidad	Proyecto de CLARISA
Materias inscritas	Science Programs (SP)
La libreta llenada a mano (guardada)	Salto A — tabla <code>bilateral_project_mapping</code>
Preguntar las materias en vivo	Salto B — función <code>deriveSciencePrograms()</code>

Entonces:

- **Salto A = "guardado" (persistido):** un admin escribe a mano "el contrato Agresso X corresponde al proyecto CLARISA Y". Queda en la base de datos porque **no hay forma automática** de saberlo; alguien tiene que decidir esa correspondencia.
- **Salto B = "en caliente" (derivado):** con el proyecto CLARISA ya en mano, el sistema le pregunta a CLARISA **en vivo** qué SP tiene. **No se guarda**: se calcula en el momento, cada vez que alguien abre la pantalla.

Por qué importa para parametrizar:

- ¿Quieres **cambiar a qué proyecto apunta un contrato**? → editas la libreta (Salto A, tabla `bilateral_project_mapping`, desde el admin).
- ¿Un SP **no aparece** aunque debería? → el problema está en el Salto B: o el proyecto en CLARISA no lo tiene confirmado, o no cumple los filtros (nivel / prefijo / portafolio). Eso **no se arregla en nuestra base de datos**, sino en CLARISA o ajustando los filtros.

Las secciones siguientes (§1.1–§1.5) son el **detalle técnico** de esos mismos dos saltos.

1.1 Salto A — persistido: `bilateral_project_mapping`

Une un **contrato de Agresso** con un **proyecto de CLARISA**.

- Entidad: `bilateral-project-mapping/entities/bilateral-project-mapping.entity.ts`
- Migración: `1779190000011-createBilateralProjectMapping.ts`

Columna	Tipo	Significado
<code>agresso_agreement_id</code>	<code>varchar(50)</code>	FK-by-value a <code>agresso_contract.agreement_id</code>

Columna	Tipo	Significado
clarisa_project_id	int	project.id de CLARISA
clarisa_project_short_name	varchar(500)	snapshot del nombre al momento del mapeo (D-PI-11)
source	enum	MANUAL por defecto
confidence_score	float	solo cuando source != MANUAL
notes	text	libre

- **Mantenimiento:** manual, desde el panel `/admin/bilateral-project-mappings` (T-15.15). No hay campo de join en upstream (D-PI-8): alguien decide qué proyecto CLARISA corresponde a cada contrato Agresso.
- **Unicidad parcial (D-PI-9):** "un solo mapeo **activo** por `agresso_agreement_id`", garantizada por una columna generada `active_agreement_id = IF(is_active=1, agresso_agreement_id, NULL)` + índice único `uk_bpm_active_agreement`. Permite histórico desactivado sin colisionar.

1.2 Salto B — en caliente: `deriveSciencePrograms()`

Los SP **no** salen de una tabla: se **derivan del proyecto CLARISA** cada vez.

- `bilateral.service.ts:446-481`

```
private deriveSciencePrograms(project): Array<{ code: string; name: string }> {
  const activePortfolio = ENV.BILATERAL_ACTIVE_PORTFOLIO; // p.ej. 'P25'
  // por cada mapping del proyecto CLARISA:
  if (
    m.status === 'Confirmed' &&
    u.level === 1 &&
    prefix === 'SP' &&
    u.portfolio_object?.acronym === activePortfolio
  ) { /* es un SP válido */ }
}
```

Las **cuatro condiciones** que un mapping del proyecto debe cumplir para considerarse un SP:

1. `status === 'Confirmed'`
2. `level === 1`
3. `prefix === 'SP'`
4. `portfolio.acronym === BILATERAL_ACTIVE_PORTFOLIO (env)`

⚠ Implicación de parametrización (ver memoria de aceleradores): programas como SP12/aceleradores que **no** cumplan `prefix==='SP'` y `level===1` quedan fuera de la lista derivada y por tanto no obtienen catálogo ToC. Si se requiere incluirlos, hay que ajustar estos filtros (o el portafolio activo), no una tabla.

1.3 Cadena completa

```

Result (result_id)
  → ResultContract (activo, primario)
    → AgressoContract.agreement_id
      → bilateral_project_mapping (fila activa) [BD, manual]
        → clarisa_project_id
          → CLARISA GET /api/projects [en caliente, cache 5 min]
            → project.project_mappings_array
              → deriveSciencePrograms() (4 filtros) [en caliente]
                → [SP01, SP02, ...]

```

1.4 Catálogo local de SP (solo display)

`clarisa_science_programs` (migración `...010`, servicio `ClarisaScienceProgramsService`) **no se sincroniza desde CLARISA**: está **sembrado a mano** y solo aporta campos de presentación (`color`, `icon_key`). La pertenencia real SP↔proyecto se decide con los filtros de §1.2.

1.5 Palancas de parametrización (Salto A/B)

Qué quieres cambiar	Dónde
Portafolio activo que define qué SP cuentan	env <code>ARI_BILATERAL_ACTIVE_PORTFOLIO</code>
Qué proyecto CLARISA corresponde a un contrato Agresso	panel <code>/admin/bilateral-project-mappings</code> (tabla <code>bilateral_project_mapping</code>)
Criterio de qué es un "SP" (prefijo/level)	<code>deriveSciencePrograms()</code> en <code>bilateral.service.ts</code>
Color/ícono de cada SP	seed/tabla <code>clarisa_science_programs</code>

2. Mapeo indicadores STAR ↔ indicadores ToC

2.1 La tabla: `result_pool_funding_toc_alignment`

- Entidad: `bilateral/entities/result-pool-funding-toc-alignment.entity.ts`
- Migración: `1779190000015--createResultPoolFundingTocAlignment.ts`
- Granularidad: **una fila activa por (`result_id`, `sp_code`)**.

Columna	Significado
<code>result_id</code> , <code>sp_code</code>	clave de negocio
<code>aligns_with_toc</code> (bool)	¿este SP alinea con la ToC?
<code>level</code>	<code>OUTPUT</code> / <code>OUTCOME</code> / <code>EOI</code>
<code>toc_result_id</code>	id del resultado ToC elegido (catálogo lambda)
<code>indicator_id</code>	id del indicador ToC dentro de ese resultado
<code>quantitative_contribution</code>	aporte cuantitativo
<code>toc_result_title</code>	snapshot del título del resultado ToC

Columna	Significado
indicator_description	snapshot de la descripción del indicador
unit_messurament	snapshot de la unidad de medida
target_value	snapshot del target
target_year	año del target (= versión live)

- Si `aligns_with_toc = false`, todas las columnas ToC/snapshot se persisten en `NULL`.
- Los **snapshots** congelan el catálogo upstream al momento de guardar (R-BIL-095): aunque la ToC en lambda cambie luego, la alineación guardada conserva lo que el usuario vio.
- Unicidad parcial (D-PI-9): `active_result_sp = IF(is_active=1, CONCAT(result_id, ':', sp_code), NULL)`
 - índice único `idx_rpfta_active_result_sp`.

2.2 Reglas de nivel por tipo de indicador STAR

Fuente única de verdad: `bilateral/utlis/toc-level-rules.util.ts`.

```
// tipo de resultado ToC por indicador STAR
capacity_sharing -> OUTPUT
innovation_dev    -> OUTPUT
knowledge_product, oicr, innovation_use, policy_change -> (mapeo de tipo)

// niveles permitidos por tipo
capacity_sharing : [OUTPUT]
innovation_dev   : [OUTPUT]
policy_change    : [OUTCOME, EOI]
```

El indicador STAR del resultado determina qué `level` ToC se le puede asignar. Esto se valida antes de guardar.

2.3 Validación al guardar: `validateTocAlignments()`

- `bilateral.service.ts:801-968`
- **Candado de versión:** el mapeo está bloqueado a `MAPPABLE_LIVE_VERSION`. Si `report_year_id` no coincide, lanza `ConflictException` con código `toc_mapping_version_locked`.
- Errores de validación posibles (R-BIL-094): `duplicate_sp_code`, `sp_not_selected`, `missing_required_fields`, `level_not_allowed`, `unknown_toc_result_id`, `unknown_indicator_id`.

2.4 Palancas de parametrización (STAR ↔ ToC)

Qué quieres cambiar	Dónde
Qué niveles ToC permite cada tipo de indicador	<code>toc-level-rules.util.ts</code>
Mapeo indicador STAR → tipo de resultado ToC	<code>toc-level-rules.util.ts</code> (<code>INDICATOR_RESULT_TYPE_KEYS</code>)
Versión ToC editable (candado)	<code>MAPPABLE_LIVE_VERSION</code>

Qué quieres cambiar	Dónde
Catálogo de resultados/indicadores ToC seleccionables	upstream lambda (no editable aquí; ver §3-4)

3. Endpoints de CLARISA en uso

Hay **tres hosts distintos**. No confundirlos.

3.1 CLARISA principal — `ARI_CLARISA_HOST`

Autenticado: hace `POST {host}auth/login` con `ARI_CLARISA_USER` / `ARI_CLARISA_PASS` y usa el bearer token. Definición de paths: `clarisa/anum/path.enum.ts`. Transporte: `clarisa/clarisa.connection.ts`.

Endpoint	Método	Uso
<code>api/countries</code> , <code>api/regions/un-regions</code> , <code>api/institutions</code> , <code>api/institution-types/simple</code> , <code>api/languages</code> , <code>api/alliance-levers</code> , <code>api/subnational-scope</code> , <code>api/sdgs</code> , <code>api/impact-areas</code> , <code>api/global-targets</code> , <code>api/sdg-targets</code>	GET	catálogos de referencia → replicados a MySQL (cron 8h)
<code>api/projects</code>	GET	proyectos bilaterales (filtra <code>source_of_funding</code> === 'Bilateral')
<code>integration/open-search/{countries,institutions,subnational}/search</code>	GET	búsqueda federada
<code>api/partner-requests/create</code>	POST	crear partner requests
<code>api/app-secrets/create</code> , <code>api/app-secrets/validate</code>	POST	permisos MIS / validar credenciales

3.2 ToC lambda — `ARI_TOC_INTEGRATION_HOST`

Valor actual: `https://clarisatest-lambda.ciat.cgiar.org`. **Sin auth** (verificado 2026-06-09).

- `GET /api/toc-integration/toc/results/category/{LEVEL}/initiative/{SP}`
- `toc-integration.service.ts:67`

3.3 PRMS ToC — `ARI_PRMS_TOC_HOST`

- `GET /api/public-results-framework/toc-results?program=<SP>&areaOfWork=<AOW>`
- `prms-toc.service.ts:68`

Aclaración importante: los **proyectos bilaterales NO se piden a `ARI_TOC_INTEGRATION_HOST`**. Salen de `ARI_CLARISA_HOST + api/projects` (apunta a

`https://api.clarisa.cgiar.org/api/projects`). `ARI_TOC_INTEGRATION_HOST` (lambda) es **solo** para el catálogo ToC.

4. ¿Replicado en BD o consultado en caliente?

Dato	¿Dónde vive?	Estrategia	Detalle
Catálogos de referencia CLARISA	MySQL	replicado	<code>cron</code> cada 8h (<code>clarisa.cron.ts</code> → <code>cloneAllClarisaEntities</code>)
Proyectos bilaterales (<code>api/projects</code>)	En caliente	cache 5 min en memoria	<code>clarisa-projects.service.ts</code> ; sirve stale si CLARISA cae, 503 en cold cache
Catálogo ToC lambda	En caliente	cache 5 min por <code>SP:level</code>	<code>toc-integration.service.ts</code> ; <code>{"response": []}</code> es catálogo vacío válido
Catálogo PRMS ToC	En caliente	cache 5 min por <code>program:AOW</code>	<code>prms-toc.service.ts</code> ; 404 = catálogo vacío válido
Mapeo proyecto bilateral ↔ Agresso	MySQL	manual admin	<code>bilateral_project_mapping</code>
Alineaciones ToC del usuario	MySQL	guardado por usuario + snapshots	<code>result_pool_funding_toc_alignment</code>
Science Programs (display)	MySQL	seed a mano	<code>clarisa_science_programs</code>

Conclusión:

- El **listado de ToC** y la **pertenencia SP↔proyecto** se consultan **en caliente** (con cache de 5 min), **no se replican** en BD.
- Lo que **sí se persiste** es el **mapeo proyecto↔Agresso** (manual) y las **decisiones de alineación del usuario** (con snapshots que congelan el catálogo en el momento de guardar).

5. Checklist para empezar a parametrizar

1. **Definir mapeos proyecto↔Agresso:** cargar filas en `bilateral_project_mapping` vía `/admin/bilateral-project-mappings` (uno activo por contrato Agresso).
2. **Confirmar portafolio activo:** revisar `ARI_BILATERAL_ACTIVE_PORTFOLIO` para que los SP esperados pasen los filtros de `deriveSciencePrograms` .
3. **Revisar reglas STAR↔ToC:** confirmar `toc-level-rules.util.ts` (tipos e indicadores nuevos → añadir su mapeo y niveles permitidos).
4. **Verificar hosts/credenciales:** `ARI_CLARISA_HOST` (+ user/pass), `ARI_TOC_INTEGRATION_HOST` , `ARI_PRMS_TOC_HOST` .
5. **Validar versión editable:** `MAPPABLE_LIVE_VERSION` debe coincidir con el `report_year_id` con el que se va a mapear, o el guardado se bloquea.